

Investigation of electrons tunnelling through potential barriers

F132357 - 12/11/2023

Throughout this notebook, I will go through how electrons tunnel through different potential barriers and how the wavefunctions, transmission and reflection probabilities describe quantum tunnelling.

To describe the motion of the electron and how it behaves in different regions, I use the Time Independent Schrodinger Equation (TISE) however I will look at this for a constant potential U :

$$-\frac{\hbar^2}{2m} \frac{d^2\psi(x)}{dx^2} + U\psi(x) = E\psi(x) \quad (1)$$

The general solution gives the wavefunction showing how the electron behaves in different regions, with different potentials (U) in each region:

$$\psi(x) = A\exp(ikx) + B\exp(-ikx) \quad (1)$$

A and B are coefficients that depend on the initial conditions. And the wavevector, k :

$$k = \frac{\sqrt{2m(E - U)}}{\hbar} \quad (1)$$

Wavefunctions for two regions

For this case there are two regions, with two separate potentials that are constant, U_1 and U_2 , for their respective regions **1** and **2**. Due to the separate potentials, as shown in the relations of the wavevector and potential, there are separate wavevectors and in turn separate wavefunctions. The functions operate in two different domains (Regions) R_1 and R_2 . Take the boundary value between regions **1** and **2** to be $x = a$.

$$x < a = R_1$$

$$\psi_1(x) = A_1 \exp(ik_1 x) + B_1 \exp(-ik_1 x) \quad (1)$$

$$x > a = R_2$$

$$\psi_1(x) = A_2 \exp(ik_2 x) + B_2 \exp(-ik_2 x)$$

The transfer matrix method

The coefficients A_2 and B_2 are values put in when initialising the wavefunction. The terms with the coefficients A and B define the components of the wavefunction going left to right and right to left respectively. In the case of region 2 as there are no more subsequent boundaries there will be no reflected waves which meaning there will be no components going right to left, implying: $B_2 = 0$. To define the coefficients A_1 and B_1 , the transfer matrix method is helpful.

By equating the two wavefunctions, taking their derivatives you can relate the coefficients A_1 and B_1 to the existing A_2 and B_2 . By equating them, you ensure that the wavefunction 1 & 2 are equal at the boundary of the two regions. Taking the derivatives ensures the transition between the regions is smooth:

$$T_{1,2} = \frac{1}{2k_1} \begin{pmatrix} (k_1 + k_2) \exp(-ia(k_1 - k_2)) & (k_1 - k_2) \exp(-ia(k_1 + k_2)) \\ (k_1 - k_2) \exp(ia(k_1 + k_2)) & (k_1 + k_2) \exp(ia(k_1 - k_2)) \end{pmatrix}$$

$$\begin{pmatrix} A_1 \\ B_1 \end{pmatrix} = T_{1,2} \begin{pmatrix} A_2 \\ B_2 \end{pmatrix} \quad (1)$$

The transfer matrix method is efficient, but it works less beyond one dimensional systems, therefore it cannot accurately be related to real world situations. It works well for situations such a thin layered film, but it assumes no thickness. Moreover, it's commonly used for the (TISE) and less so for Time Dependent Schrodinger equation (TDSE).

I have defined the different packages I use to create the plots and the different functions I use from the Statistics package. The plotly and gr backends are for analysing and outputting the graphs.

```
GRBackend()
```

```
1 using Plots ; using PlutoUI ;using Statistics ; plotly() ; gr()
```

I then defined constants that I used throughout, i , $\hbar$ and the mass of an electron as variables used throughout

9.11e-31

```
1 i = 1im ; hbar = (6.63*10^(-34))/(2*pi) ; m = 9.11*10^(-31)
```

The following function is to ensure that arrays are fully defined (i.e. not including an infinity or NaN elements) as a form of testing the function when building it to find errors. It will not be used throughout the examples as they have already been tested.

isArrayDefined (generic function with 1 method)

```
1 function isArrayDefined(arr)
2     return !any(isnan.(arr)) && !any(isinf.(arr))
3 end
```

This function is for an arbitrary transfer matrix between two arbitrary regions (R_i and R_{i+1}), where the wavevectors k_i and k_{i+1} are the wavevectors for the respective regions. And the x_i is the boundary values between the regions:

$$T_{i,i+1} = \frac{1}{2k_i} \begin{pmatrix} (k_i + k_{i+1})\exp(-ia(k_i - k_{i+1})) & (k_i - k_{i+1})\exp(-ia(k_i + k_{i+1})) \\ (k_i - k_{i+1})\exp(ia(k_i + k_{i+1})) & (k_i + k_{i+1})\exp(ia(k_i - k_{i+1})) \end{pmatrix} \quad (1)$$

transferMatrix (generic function with 1 method)

```
1 function transferMatrix(kiplus1,ki,xi)
2     (1/(2*ki))*[(ki+kiplus1)*exp(-i*xi*(ki-kiplus1)) (ki - kiplus1)*exp(-
        i*xi*(ki + kiplus1)) ; (ki - kiplus1)*exp(i*xi*(ki + kiplus1)) (ki +
        kiplus1)*exp(i*xi*(ki - kiplus1))]
3 end
```

This function is to convert between eV and J units :

evtoJ (generic function with 1 method)

```
1 function evtoJ(ev::Float64)
2     return ev*1.6*10^(-19)
3 end
```

Finally, this function is to define the wavevector for an arbitrary region. With these defined the wavefunctions for the different scenarios can be constructed.

k (generic function with 1 method)

```
1 function k(m::Float64,E::Float64,U::Float64)
2     sqrt(Complex(2*m*(E - U)))/(hbar)
3 end
```

1. Potential step (Two regions)

The function starts off by defining the wavevectors k_1 and k_2 using the wavevector function previously defined with the initialised values of U_1 and U_2 . I then define 'deltax' which is the width of each region. The dx value is an arbitrary value that controls how many equidistance points there are between the boundary values that enclose the two regions defined by R_1 and R_2 .

Using the defined wavevectors, and the previously defined transfer matrix function, the transfer matrix acts on the vector of the A_2 and B_2 coefficients and produces the coefficients of A_1 and B_1 . Now using the the coefficients, wavevectors and the domains of the regions **1** and **2** to obtain an array of ψ_1 and ψ_2 to plot against their respective domains R_1 and R_2 .

The transmission and reflection coefficients determine the probability that the electron goes through the potential boundary between the regions.

Transmission coefficient :

$$Tr = \left| \frac{1}{t_{11}} \right|^2 \frac{k_N}{k_1}$$

Reflection coefficient :

$$R = \left| \frac{t_{21}}{t_{11}} \right|^2 \quad (1)$$

Where t_{ij} are the elements of the transfer matrix and k_N is the wavevector of the final region N

waveFunction (generic function with 1 method)

```

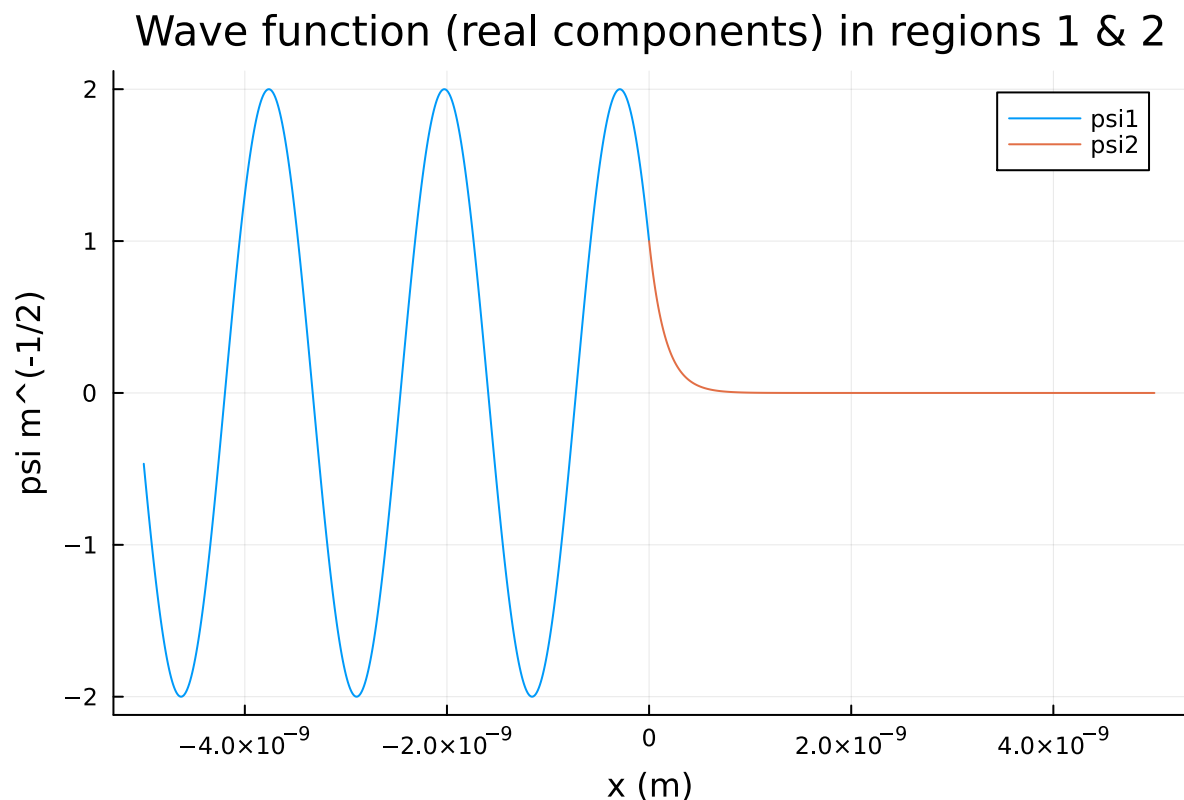
1 function
  waveFunction(m::Float64,U1::Float64,U2::Float64,E::Float64,A2::Float64,B2::Float
    64,a::Float64,RorI::String)
2     k1 = k(m,E,U1) #wavevector in region 1
3     k2 = k(m,E,U2) #wavevector in region 2
4
5     deltax = 5*10^(-9)
6     dx = 0.01*10^(-9)
7     #Region 1 x<a
8     R1 = (a*10^(-9)-deltax):dx:a*10^(-9)
9     #Region 2 x>a
10    R2 = a*10^(-9):dx:(a*10^(-9) + deltax)
11
12    T12 = transferMatrix(k2, k1, a)
13
14    #Transmission and reflection coefficients
15    Tr = (1/T12[1][1])^(2) * (k2/k1)
16    R = (T12[2][1]/T12[1][1])^(2)
17
18    println("Transmission =") ; println(real(Tr))
19    println("Reflection =") ; println(real(R))
20
21    A2B2 = [A2 ; B2]
22    A1B1 = T12 * A2B2
23    A1 = A1B1[1]
24    B1 = A1B1[2]
25
26    #Region 1 when x<a
27    psi1 = A1.*exp.(i*k1.*R1) .+ B1.*exp.(-i*k1.*R1)
28    #Region 2 when x>a
29    psi2 = A2.*exp.(i*k2.*R2) .+ B2.*exp.(-i*k2.*R2)
30
31    if RorI == "Real"
32        #real compoenents
33        plot(R1 ,real(psi1) ,xlabel= "x (m)",ylabel = "psi m^(-1/2)", label=
34        "psi1" )
35        plot!(R2 ,real(psi2) ,xlabel= "x (m)",ylabel = "psi m^(-1/2)",
36        label= "psi2")
37        plot!(title="Wave function (real components) in regions 1 & 2")
38
39    elseif RorI == "Imag"
40        #imaginary components
41        plot(R1 ,imag(psi1) ,xlabel= "x (m)",ylabel = "psi m^(-1/2)", label=
42        "psi1" )
43        plot!(R2 ,imag(psi2) ,xlabel= "x (m)",ylabel = "psi m^(-1/2)",
44        label= "psi2")
45        plot!(title="Wave function (imaginary components) in regions 1 & 2")
46
47    elseif RorI != "Real" || "Imag"
48        md"##### Input either the string 'Real' or 'Imag' to plot either the
49        real or imaginary components of the wavefunction"
50    end
51 end

```

Syntax to call the function: `waveFunction(m, U1, U2, E, A2, B2, a, RorI)`

The wavefunction defined above takes in the following arguments the mass of the give particle, the constant potentials in the regions **1** & **2**, the energy of the particle, the coefficients of region 2 A_2 and B_2 , the boundary value between the regions 'a' and the string 'RorI' which links to an if statement that plots either the real or imaginary components of the wavefunction

To initialise the function I plotted **(1.1)** the behaviour of an electron incident on a potential barrier where the energy of the electron ($E = 0.5eV$) is less than the potential ($U_2 = 2.0eV$) of the barrier. As expected the electron is incident and propagates in Region **1** and it begins to decay once it hits the potential barrier and enters Region **2**.



```

Transmission =
1.5000000000000002
Reflection =
-0.5

```



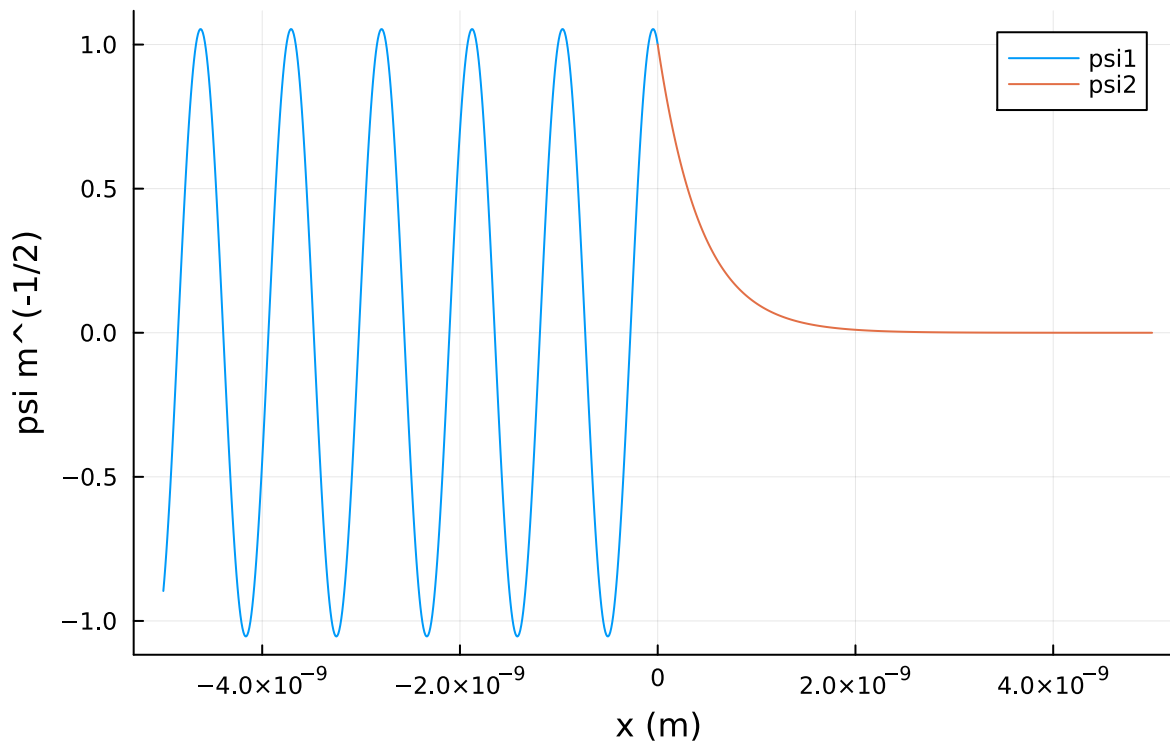
(Figure 1.1)

When increasing the energy of the incident electron (e.g. to $E = 1.8eV$ [as shown below]) yet not greater than the potential energy of the potential step, the wavelength of the incident electron decreases showing more oscillations in Region **1**. However, interestingly the point of decay stays constant despite the changing of the incident electron's energy. This implies that the two following expressions do not depend on the energy of the incident particle

$$\psi_1(a) = \psi_2(a)$$

$$\frac{d\psi_1}{dx}(a) = \frac{d\psi_2}{dx}(a)$$

Wave function (real components) in regions 1 & 2



```

Transmission =
0.72000000000000001
Reflection =
0.27999999999999997

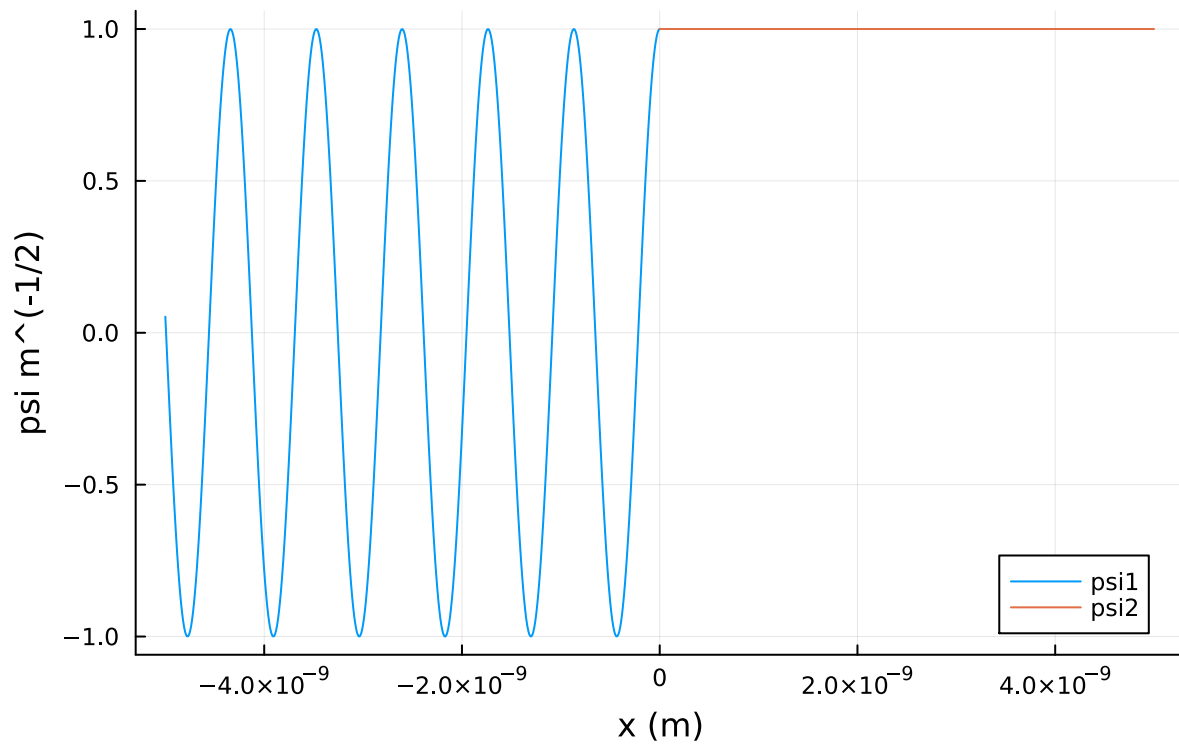
```



(Figure 1.2)

When energy of the incident electron is equal to the energy of the potential barrier $E = U_2$ (shown in **1.3**), the transmission coefficient and the reflection coefficient are **0** and **1** respectively. This means there's complete reflection, and no transimission. This phenomena is called resonance/particle trapping. Which as the name suggests traps the particle within a region (in this case Region **1**).

Wave function (real components) in regions 1 & 2



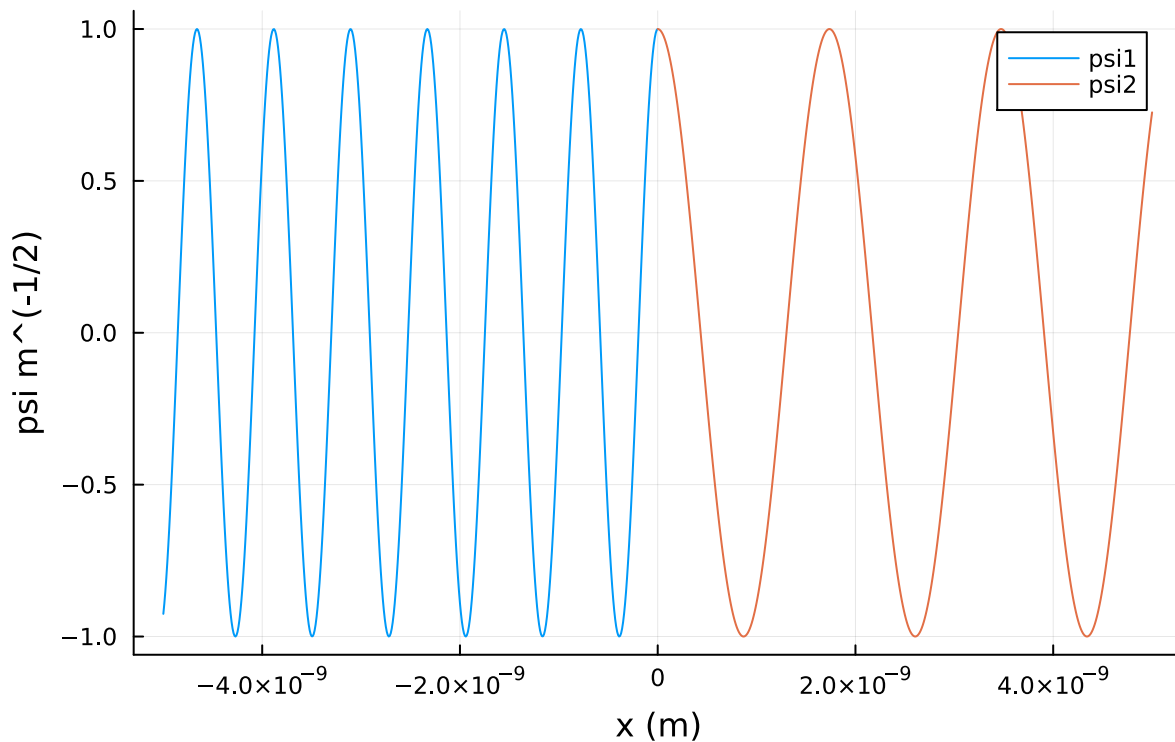
```
Transmission =  
0.0  
Reflection =  
1.0
```



(Figure 1.3)

Finally, as expected when the energy of the electron is greater than that of the potential step it oscillates beyond the barrier (with less potential energy) and does not decay as shown (1.4):

Wave function (real components) in regions 1 & 2



```

Transmission =
0.8541019662496846
Reflection =
0.14589803375031557
  
```



(Figure 1.4)

2. Single Finite Barrier (Three regions)

In order to model the transmission probability as a function of a variable energy (of the electron), modelled the potential step in three regions. The code is for the most part the same as for the potential step, upscaled for three regions. The changes being two transfer matrices on that takes the coefficients from region 3 to 2 and then region 2 to 1. As opposed to the boundary value between the two regions begin A there are now two x_1 and x_2 which are the boundaries between regions 1 & 2 and 2 & 3 respectively.

To model this as a potential step as before U_2 is the same constant potential as before and U_1 is 0 as done before, U_3 is also 0. The main difference is the introduction of $E1$ which is the range of values upon which $T(E)$ is defined, a which is the number of elements that $E1$ and $T(E)$ have. The 'ones' initializes an array of ones with a elements to overwrite to with calculated values.

As the energy of the incident electron changes there are three cases that change how it relates with the transmission probability. The δ is the width of the region. From [1] Page 38:

$$k = \sqrt{2mE/\hbar^2}$$

when: $E < V$

$$\kappa = \sqrt{2m(V - E)/\hbar^2} \quad (1)$$

$$T(E) = \frac{1}{1 + \frac{1}{4} \left(\frac{\kappa}{k} + \frac{k}{\kappa} \right)^2 \sinh^2 \kappa \delta}$$

when: $E = V$

$$\frac{1}{1 + \left(\frac{k\delta}{2} \right)^2} \quad (1)$$

when: $E > V$

$$k' = \sqrt{2m(E - V)/\hbar^2} \quad (1)$$

$$T(E) = \frac{1}{1 + \frac{1}{4} \left(\frac{k'}{k} - \frac{k}{k'} \right)^2 \sin^2 k' \delta}$$

```

1  #=
2      #The energy dependence of the transmission (Tr) probability [T(E)]
3      dE = evtToJ(0.01)
4      E1 = dE:dE:evtToJ(20.0)
5      a = Int(E1[end]/dE)
6      TE = ones(a)
7
8      for i in 1:a
9          kappa = i*k(m,E1[i],U2)
10         kprime = k(m,E1[i],U2)
11         ka = k(m,E1[i],0.0)
12
13         if E1[i] < U2
14             TE[i] = 1/(1 + (0.25)*(kappa/ka + ka/kappa)^(2)*
15             (sinh(kappa*deltax))^2)
16         elseif E1[i] == U2
17             TE[i] = 1/(1 + (ka*deltax/2)^2)
18         elseif E1[i] > U2
19             TE[i] = 1/(1 + (0.25)*(kprime/ka - ka/kprime)^(2)*
20             (sin(kprime*deltax))^2)
21         end
22     end
23
24  =#

```

waveFunction2 (generic function with 1 method)

```

1 function
  waveFunction2(m::Float64,U1::Float64,U2::Float64,U3::Float64,E::Float64,A3::Float64,B3::Float64,x1::Float64,RorIorTr::String)
2     deltax = 5*10^(-9)
3     #Wavevectors in each region
4     k1 = k(m,E,U1) ; k2 = k(m,E,U2) ; k3 = k(m,E,U3)
5     #Boundary values
6     x1 = x1*10^(-9) ; x2 = x1 + deltax
7
8     dx = (0.01*10^(-9))
9
10    #Region 1
11    R1 = (x1-deltax):dx:x1
12    #Region 2
13    R2 = x1:dx:x2
14    #Region 3
15    R3 = x2:dx:(x2+deltax)
16
17    #Defining the coefficients of the wavefunctions
18    T12 = transferMatrix(k2, k1, x1)
19    T23 = transferMatrix(k3, k2, x2)
20
21    A3B3 = [A3 ; B3]
22    A2B2 = T23 * A3B3
23    A1B1 = T12 * A2B2
24    A1 = A1B1[1] ; B1 = A1B1[2] ; A2 = A2B2[1] ; B2 = A2B2[2]
25
26    #when Region 1 (0<R1<x1)
27    psi1 = A1.*exp.(i*k1.*R1) .+ B1.*exp.(-i*k1.*R1)
28    #when Region 2 (x1<R2<x2)
29    psi2 = A2.*exp.(i*k2.*R2) .+ B2.*exp.(-i*k2.*R2)
30    #when Region 3 (x2<R3<(x2+deltax))
31    psi3 = A3.*exp.(i*k3.*R3) .+ B3.*exp.(-i*k3.*R3)
32
33    #The energy dependence of the transmission (Tr) probability [T(E)]
34    dE = evtoJ(0.01)
35    E1 = dE:dE:evtoJ(20.0)
36    a = Int(E1[end]/dE)
37    TE = ones(a)
38
39    for i in 1:a
40        kappa = i*k(m,E1[i],U2)
41        kprime = k(m,E1[i],U2)
42        ka = k(m,E1[i],0.0)
43
44        if E1[i] < U2
45            TE[i] = 1/(1 + (0.25)*(kappa/ka + ka/kappa)^(2)*
46            (sinh(kappa*deltax))^2)
47        elseif E1[i] == U2
48            TE[i] = 1/(1 + (ka*deltax/2)^2)
49        elseif E1[i] > U2
50            TE[i] = 1/(1 + (0.25)*(kprime/ka - ka/kprime)^(2)*
51            (sin(kprime*deltax))^2)
52        end
53    end
54
55    if RorIorTr == "Real"
```

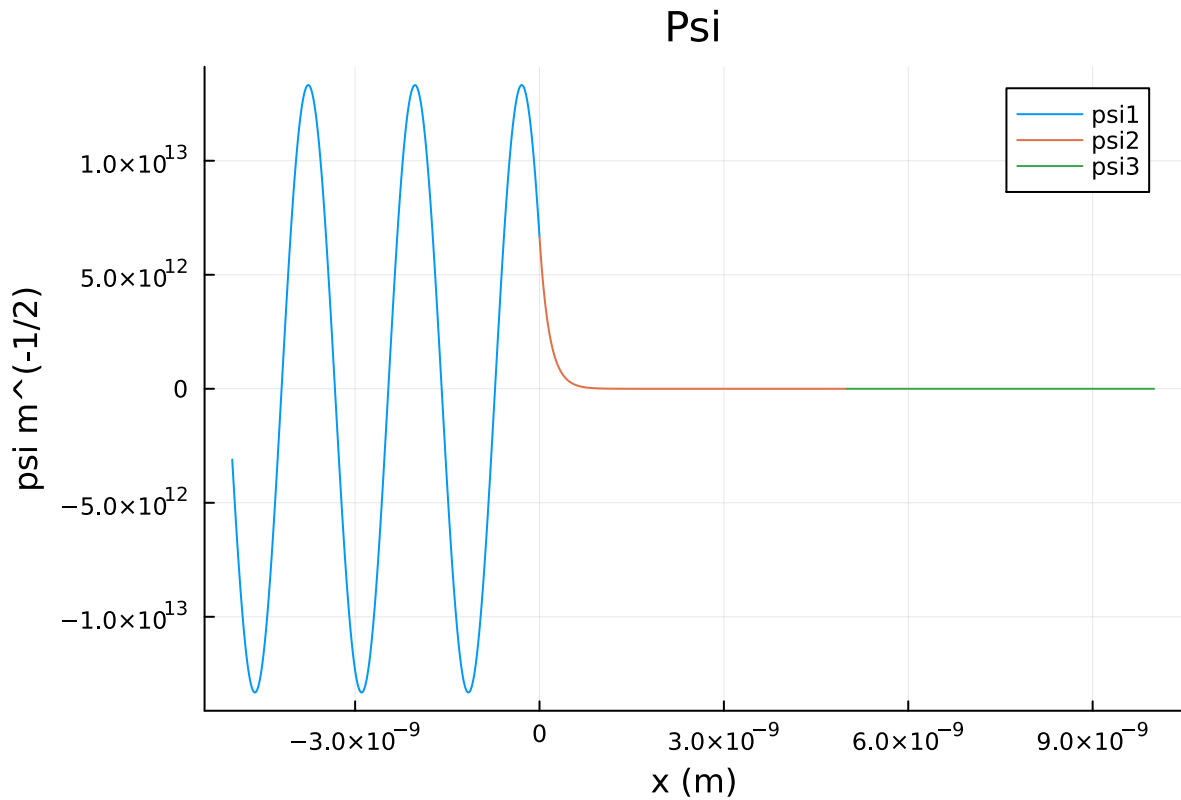
```

56         #real compoenents
57         plot(R1 ,real(psi1) ,xlabel= "x (m)",ylabel = "psi m^(-1/2)", label=
58         "psi1" )
59         plot!(R2 ,real(psi2) ,xlabel= "x (m)",ylabel = "psi m^(-1/2)",
60         label= "psi2")
61         plot!(R3 ,real(psi3) ,xlabel= "x (m)",ylabel = "psi m^(-1/2)",
62         label= "psi3")
63         plot!(title="Psi")
64
65     elseif RorIorTr == "Imag"
66         #imaginary components
67         plot(R1 ,imag(psi1) ,xlabel= "x (m)",ylabel = "psi m^(-1/2)", label=
68         "psi1" )
69         plot!(R2 ,imag(psi2) ,xlabel= "x (m)",ylabel = "psi m^(-1/2)",
70         label= "psi2")
71         plot!(R3 ,imag(psi3) ,xlabel= "x (m)",ylabel = "psi m^(-1/2)",
72         label= "psi3")
73         plot!(title="Psi")
74
75     elseif RorIorTr == "Tr"
76         plot(E1, TE,xlabel= "E (J)",ylabel = "Tr(E[J])", label= "T(E)")
77         plot!(title="Transmission Probability")
78     elseif RorIorTr != "Real" || "Imag" || "Tr"
79         md"##### Input either the string 'Real' or 'Imag' to plot either the
80         real or imaginary components of the wavefunction. And to plot the
81         transmission as a function of energy, 'Tr'."
82     end
83
84 end

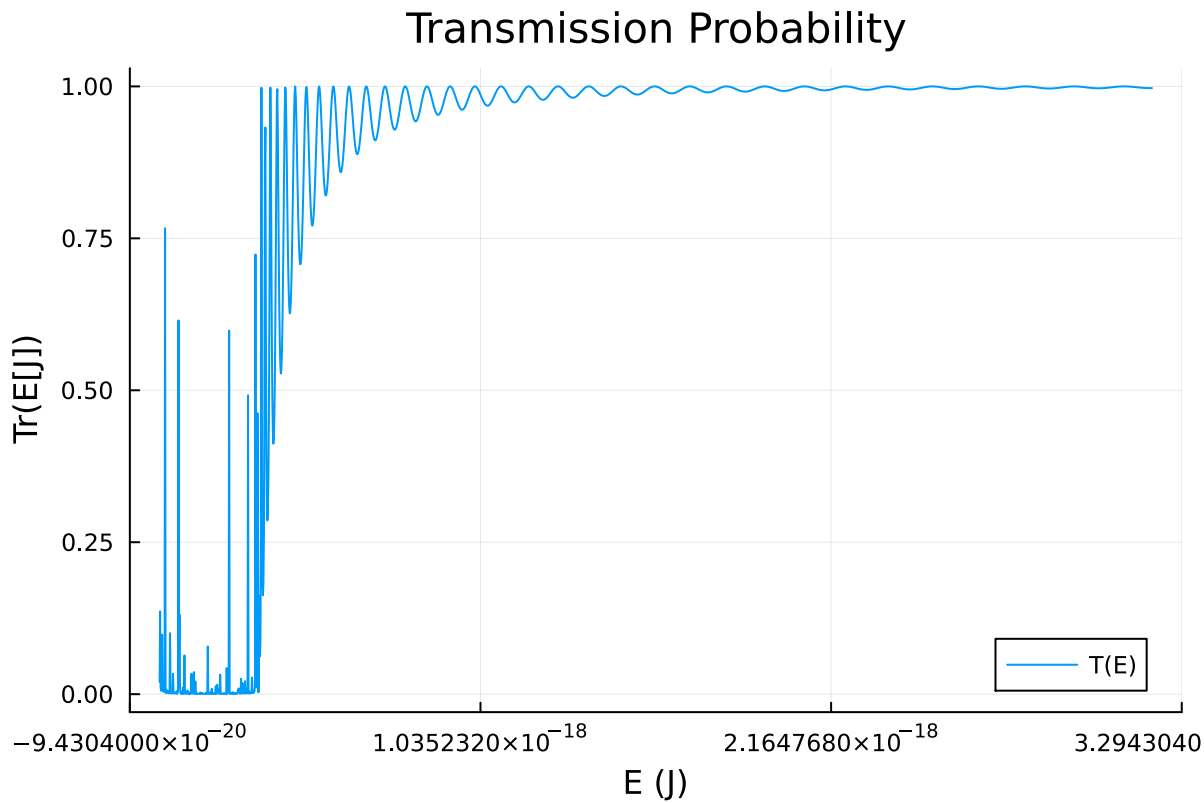
```

Syntax to call the function: *waveFunction2*(*m*, *U*₁, *U*₂, *U*₃, *E*, *A*₃, *B*₃, *a*, *RorIorTr*)

With these initial conditions, the finite barrier outputs the same wavefunction as for the potential step. See, (2.1) :



(Figure 2.1)



(Figure 2.2)

The graph above (2.2) shows how the transmission probability and how it changes for a finite barrier. The shape follows the expected outcome, where the probability increases as the energy increases as it oscillates. If you were to fit a curve through the troughs of the oscillation you would obtain the lower bound of transmission probability and if you were to fit a curve to the peaks you would have the upper bounds of the transmission probability. The also lines up with the literature when compared with the same initial conditions [1] -Page 39 (Figure 10).

3. Potential step : (Five regions) [Double Barrier]

The following function is for a double barrier system of an electron incident travelling from left to right. Defining the distance between the two barriers w , width of both barriers b , start of the system's domain x_0 and the potential of each barrier U_1 and U_2 respectively.

The code follows a similar structure upscaled to the necessary five regions. Where R_1 is the first region of an arbitrary width a and $U = 0.0\text{eV}$, followed by R_2 which contains the first barrier with a width b , then R_3 which is the gap between the two barriers of width w with the potential as U . Finally, R_4 is the same as R_2 except with a potential of U_2 and R_5 is the same as R_1 just on the other side (they act as reference regions to see how the particle enters the double barrier system [in R_1] and leaves the double barrier [in R_5]).

As the potentials of regions 1, 3 & 5 are all zero (U), they all have the same wavevector defined by k_i [shown below]. To create a continuous wavefunction, showing the system in its entirety, the boundary values have to be defined relative to the start of systems domain x_0 which is 0.0 for ease of understanding in my plots. Finally, dx is used to define the equidistant data points in each region.

```

1  #=
2      #Potential and wavevector in Regions 1,3 & 5 (No potential)
3      U = 0.0 ; ki = k(m,E,U)
4      #Wavevectors in Regions 2 & 4 (Barrier regions)
5      k1 = k(m,E,U1) ; k2 = k(m,E,U2)
6
7      #Boundary values
8      x1 = a ; x2= a+b ; x3= a+b+w ; x4= a+2*b+w ; x5= 2*a+2*b+w
9
10     #Defining each region
11     dx = 0.01 * 10^(-9)
12     R1 = x0:dx:x1 ; R2 = x1:dx:x2 ; R3 = x2:dx:x3 ; R4 = x3:dx:x4 ; R5 =
13     x4:dx:x5
14     =#

```

As explained in Section 2 the final region has coefficients of 1.0 and 0.0 , which in this case are the coefficients A_5 and B_5 respectively. To obtain the coefficients in each region I define the transfer matrices once again (this time 4 of them as there are 5 regions). Using those coefficients and the regions, the wavefunctions for each region is formed.

The code section for the energy dependence of the transmission probability ' $Tr(E)$ ' has not changed, and the if statement '*Real**or**Tr*' is for the input string which lets you plot either the 'Real', 'Imag' or 'Tr' as done in the previous two systems.

waveFunction3 (generic function with 1 method)

```

1 function
  waveFunction3(m::Float64,U1::Float64,U2::Float64,E::Float64,a::Float64,b::Float64,w::Float64,A5::Float64,B5::Float64,x0::Float64,RorIorTr::String)
2     #Potential and wavevector in Regions 1,3 & 5 (No potential)
3     U = 0.0 ; ki = k(m,E,U)
4     #Wavevectors in Regions 2 & 4 (Barrier regions)
5     k1 = k(m,E,U1) ; k2 = k(m,E,U2)
6
7     #Boundary values
8     x1 = a ; x2= a+b ; x3= a+b+w ; x4= a+2*b+w ; x5= 2*a+2*b+w
9
10    #Defining each region
11    dx = 0.01 * 10^(-9)
12    R1 = x0:dx:x1 ; R2 = x1:dx:x2 ; R3 = x2:dx:x3 ; R4 = x3:dx:x4 ; R5 =
13    x4:dx:x5
14
15    #Defining the coefficients of the wavefunctions
16    T12 = transferMatrix(k1,ki,x1)
17    T23 = transferMatrix(ki,k1,x2)
18    T34 = transferMatrix(k2,ki,x3)
19    T45 = transferMatrix(ki,k2,x4)
20
21    A5B5 = [A5 ; B5]
22    A4B4 = T45*A5B5 ; A3B3 = T34*A4B4 ; A2B2 = T23*A3B3 ; A1B1 = T12*A2B2
23    A1=A1B1[1] ; B1=A1B1[2] ; A2=A2B2[1] ; B2=A2B2[2] ; A3=A3B3[1] ;
24    B3=A3B3[2]
25    A4=A4B4[1] ; B4=A4B4[2] ; A5=A5B5[1] ; B5=A5B5[2]
26
27    #Defining the wavefunctions in each region
28    psi1 = A1.*exp.(i*ki.*R1) .+ B1.*exp.(-i*ki.*R1)
29    psi2 = A2.*exp.(i*k1.*R2) .+ B2.*exp.(-i*k1.*R2)
30    psi3 = A3.*exp.(i*ki.*R3) .+ B3.*exp.(-i*ki.*R3)
31    psi4 = A4.*exp.(i*k2.*R4) .+ B4.*exp.(-i*k2.*R4)
32    psi5 = A5.*exp.(i*ki.*R5) .+ B5.*exp.(-i*ki.*R5)
33
34    #The energy dependence of the transmission (Tr) probability [Tr(E)]
35    dE = evtoJ(0.01)
36    E1 = dE:dE:evtoJ(20.0)
37    a = Int(E1[end]/dE)
38    TE = ones(a)
39
40    for i in 1:a
41        kappa = k(m,E1[i],U2)
42        kprime = k(m,E1[i],U2)
43        ka = k(m,E1[i],0.0)
44
45        if E1[i] < U2
46            TE[i] = 1/(1 + (0.25)*(kappa/ka + ka/kappa)^(2)*
47            (sinh(kappa*b))^2)
48        elseif E1[i] == U2
49            TE[i] = 1/(1 + (ka*b/2)^2)
50        elseif E1[i] > U2
51            TE[i] = 1/(1 + (0.25)*(kprime/ka - ka/kprime)^(2)*
52            (sin(kprime*b))^2)
53        end
54    end
55

```



```

56     if RorIorTr == "Real"
57         #real components
58         plot(R1 ,real(psi1) ,xlabel= "x (m)",ylabel = "psi m^(-1/2)", label=
59 "psi1" )
60         plot!(R2 ,real(psi2) ,xlabel= "x (m)",ylabel = "psi m^(-1/2)",
61 label= "psi2")
62         plot!(R3 ,real(psi3) ,xlabel= "x (m)",ylabel = "psi m^(-1/2)",
63 label= "psi3")
64         plot!(R4 ,real(psi4) ,xlabel= "x (m)",ylabel = "psi m^(-1/2)",
65 label= "psi4")
66         plot!(R5 ,real(psi5) ,xlabel= "x (m)",ylabel = "psi m^(-1/2)",
67 label= "psi5")
68         plot!(title="Psi")
69
70     elseif RorIorTr == "Imag"
71         #imaginary components
72         plot(R1 ,imag(psi1) ,xlabel= "x",ylabel = "psi", label= "psi1" )
73         plot!(R2 ,imag(psi2) ,xlabel= "x",ylabel = "psi", label= "psi2")
74         plot!(R3 ,imag(psi3) ,xlabel= "x",ylabel = "psi", label= "psi3")
75         plot!(R4 ,imag(psi4) ,xlabel= "x",ylabel = "psi", label= "psi4")
76         plot!(R5 ,imag(psi5) ,xlabel= "x",ylabel = "psi", label= "psi5")
77         plot!(title="Psi")
78
79     elseif RorIorTr == "Tr"
80         plot(E1, TE,xlabel= "E(J)",ylabel = "Tr(E[J])", label= "T(E)")
81         plot!(title="Transmission Probability")
82
83     elseif RorIorTr != "Real" || "Imag" || "Tr"
84         md"##### Input either the string 'Real' or 'Imag' to plot either the
85 real or imaginary components of the wavefunction. And to plot the
86 transmission as a function of energy, 'Tr'."
87     end
88 end

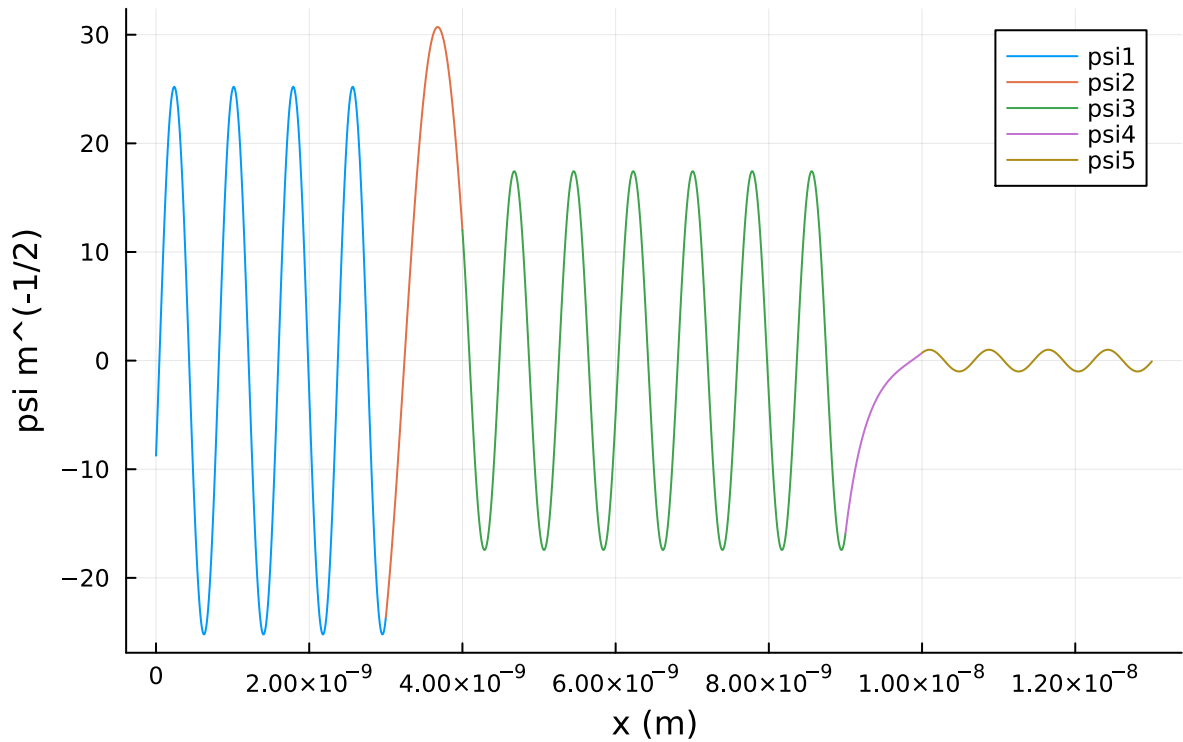
```

Syntax to call the function: *waveFunction3*(*m*, *U*₁, *U*₂, *E*, *a*, *b*, *w*, *A*₅, *B*₅, *x*₀, *RorIorTr*)

In (3.1) I initialised the wavefunction such that potential of the first barrier ' $U_1 = 2.0\text{eV}$ ' is marginally less than the energy of the particle ' $E = 2.5\text{eV}$ ', but the potential of the second barrier is marginally greater ' $U_2 = 3.0\text{eV}$ '. If you first look at the energy of the particle before reaching the first barrier in blue ' ψ_1 ' and compare it after going through the first barrier in Region 3 (green ' ψ_3 ') it's energy has decreased. This is the expected outcome (as shown in (1.4)) when the energy of the particle is marginally greater than that of the barrier.

Now as the particle continues towards the second barrier, it begins to decay into the barrier ψ_4 , as was shown in (1.1). However due to the smaller width of the potential barrier ($b = 1 * 10^{-9}$), as opposed to the one shown in (1.1), the particle does not fully decay within ψ_4 leaving the second barrier with some energy. In ψ_5 , you can see the electron propagates with some small energy, instead of levelling off.

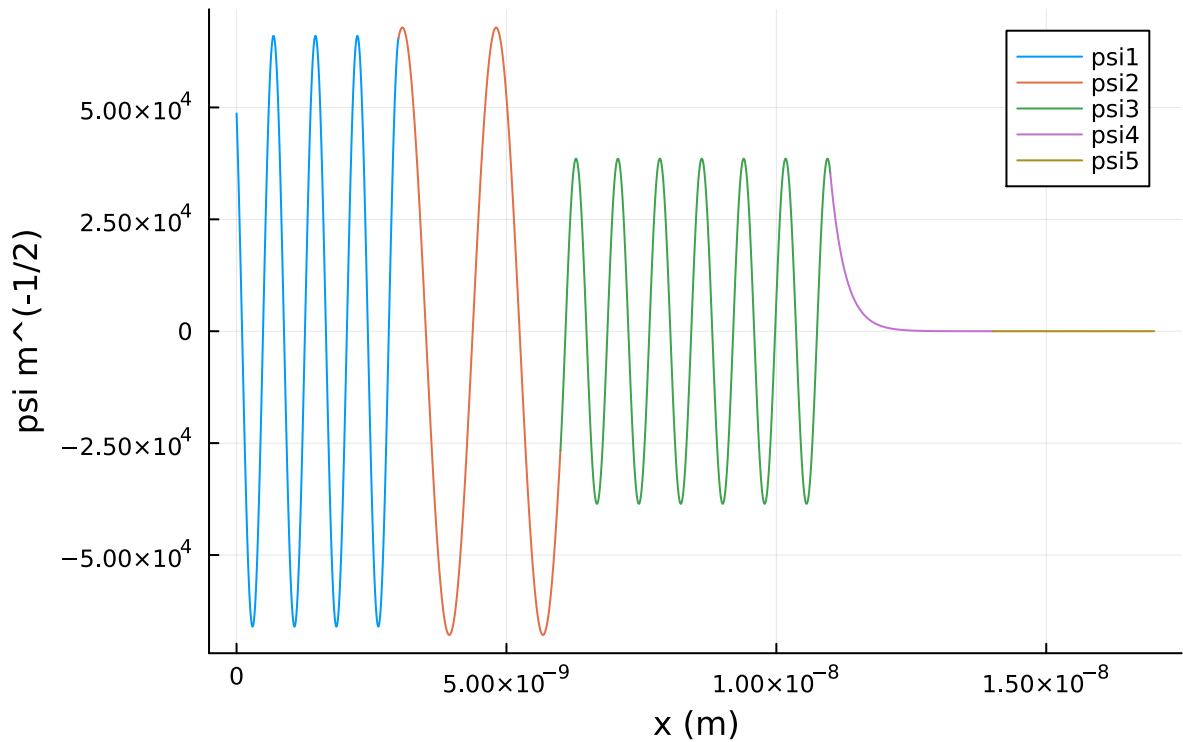
Psi



(Figure 3.1)

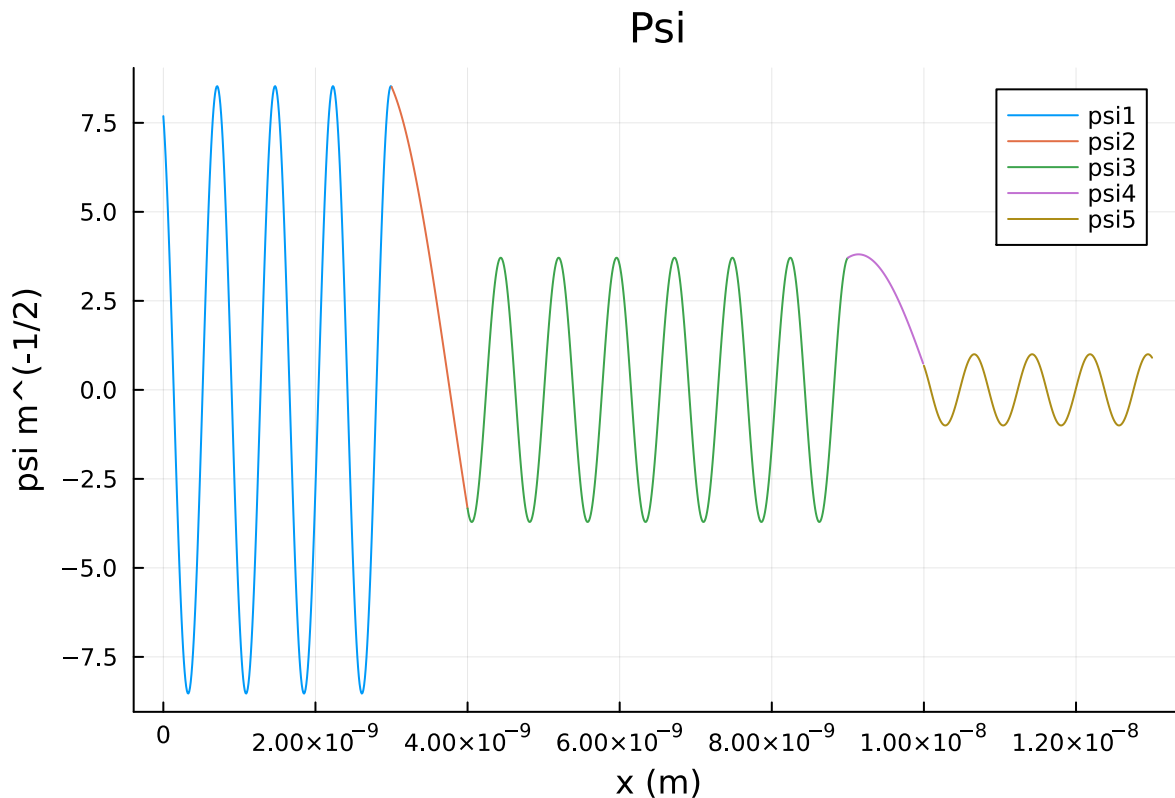
In (3.2) I tripled the width of the potentials 'b' to reaffirm the conclusion I came to. Here the particle decays further, as it has more to tunnel through, to the point of negligible energy. Clearly, here there are no oscillations is ψ_5 when compared to the same region in (3.1).

Psi



(Figure 3.2)

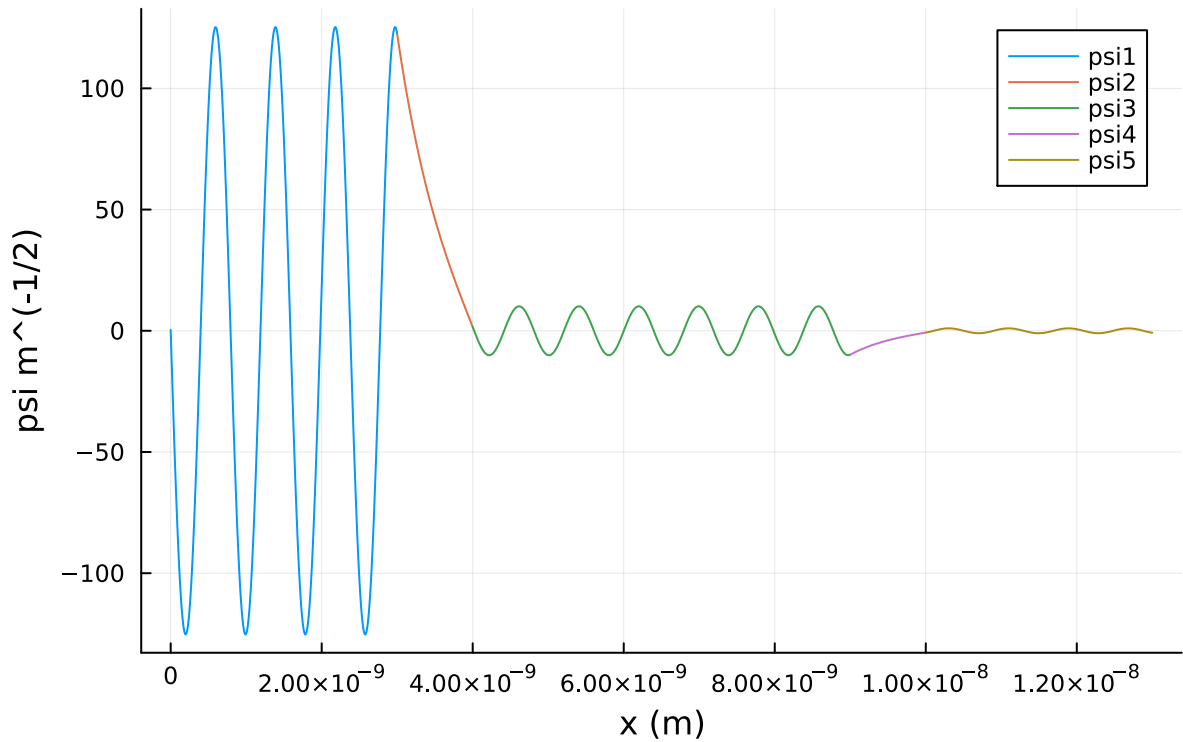
In the example below **(3.3)** I show how the system evolves when the potentials of both barriers are equal $U_1 = U_2 = 2.5\text{eV}$, but the energy of the electron is marginally greater $E = 2.6\text{eV}$ than the potential of both barriers. The energy of the electron is dampened as it goes through each barrier.



(Figure 3.3)

But when the energy of the electron is marginally smaller than the potentials $E = 2.4\text{eV}$ the affect of the dampening is far more pronouced. This demonstrates the phenomena of quantum tunneling for an electron, and how exaggerated the rate of decay is (especially in comparison to the classical case where the electron has greater energy) relative to the example **(3.3)**.

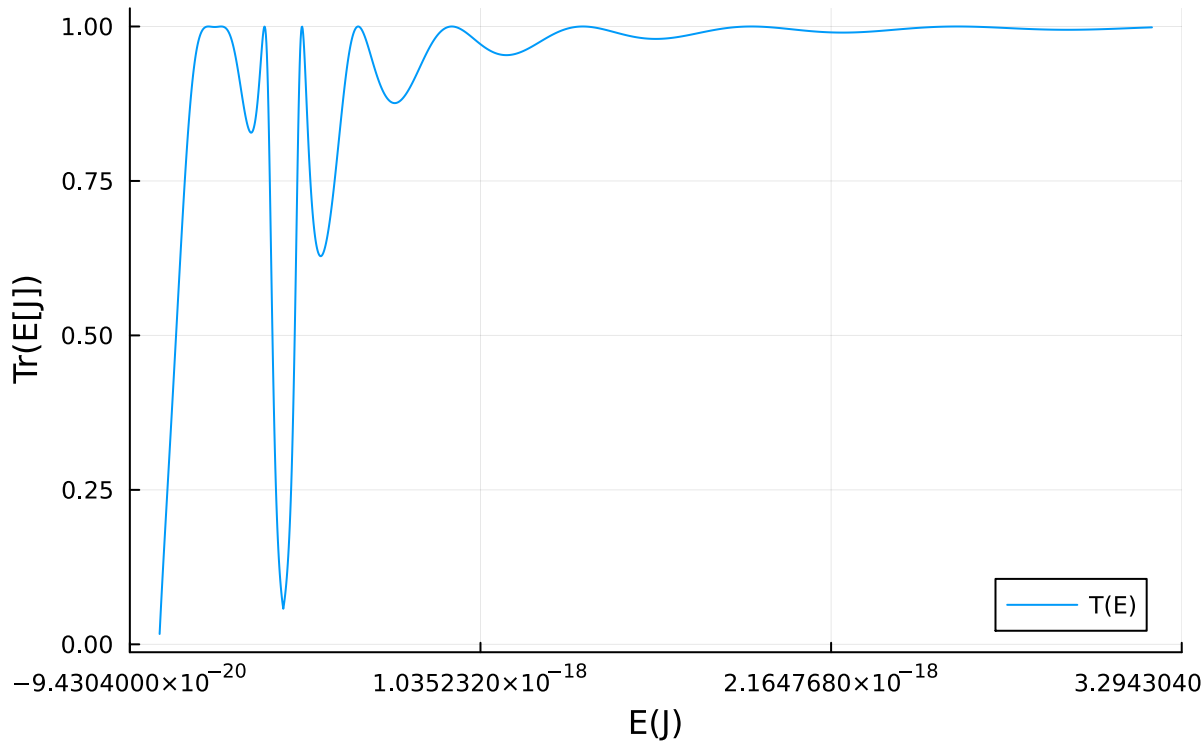
Psi



(Figure 3.4)

The graph below follows the expected pattern with the decrease in displacement of the troughs. The equations used in section 2 are analytical in nature. It is good for simple systems however as the systems begin to get more complex it begins to falter, becoming less accurate though it is still a good approximation for the double barrier.

Transmission Probability



4. Variable potential (Arbitrary number of regions)

The function below outputs a variable potential as opposed to the constant potential used in the previous two examples. The variable potential follows a Gaussian shape for the barrier. Using the equation, from [1] - Figure (116) :

$$V(x) = Ve^{-x^2} \quad (1)$$

U (generic function with 1 method)

```
1 function U(x::Float64,U::Float64)
2     U * exp(-(x^2))
3 end
```

This function truncates a float variable to an arbitrary number of significant figures. I have struggled to find an alternative that doesn't decrease the accuracy. Potentially doing this in a coding language better suited to numerical solutions with maths in mind would let me side step this function. Similarly, a suitable package would also be useful.

truncateToSf (generic function with 1 method)

```
1 function truncateToSf(x::Float64, sf::Int)
2     if x != 0.0
3         # Calculate the order of magnitude
4         order_magnitude = floor(log10(abs(x)))
5         # Calculate the scaling factor based on significant figures
6         scale_factor = 10^(sf - 1 - order_magnitude)
7         # Truncate the scaled number
8         truncated_number = trunc(x * scale_factor) / scale_factor
9         return truncated_number
10    else
11        return x
12    end
13 end
```

Wavefunction **3** here takes the coefficients of the final region N , (A_N and B_N), which are used to compute the coefficients of the different regions. The x_1 and x_{n-1} are the first and last boundary values. N , the number of regions. The lowercase v is the value for an arbitrarily chosen region which will output the wavefunction at the boundary of regions $v - 1$ and v . The uppercase V is to select the coefficient V for the variable potential defined in the equation for the Gaussian barrier above.

The if statement below ensures the selected the region v is not beyond the total number or regions N selected. And as mentions previously when selecting an arbitrary region v the wavefunction if plotted at the boundary of $v - 1$ and v . There it must be 2 or greater as $v-1$ has to be greater than or equal to 1. Similarly, N has to be at least **2** for it to be a non trivial case.

```

1  #=
2      if v > N
3          return md"#### The specified region 'v' to create the wavefunction
           is more than the total number of regions"
4      elseif v == 1
5          return md"#### The specified region 'v' to create the wavefunction
           has to be greater than 1 to compare it to a region one less"
6      end
7  =#

```

Below I defined the region boundaries where deltax is the constant width of each region. Due to the nature of Julia there is some inaccurate rounding which is solved using the earlier defined truncating function. The arbitrary boundary values between regions is defined by BR_i . Each region has a set number of x data points defined by y . R_i is an $N * y$ matrix where each array for gives the domain for each region. Shown by the for loop.

```

1  #=
2      deltax = (xsub1 - x1)/(N-2)
3      deltax = truncateToSf(deltax, 3)
4
5      #Region boundaries
6      BRi = (x1 - deltax):deltax:(xsub1 + deltax)
7
8      #Creation of regions
9      dx = 0.01*10^(-9)
10     y = Int(round(((BRi[2] - BRi[1])/dx)) + 1)
11     Ri = ones(Float64, N , y)
12     for i in 1:(N)
13         Ri[i, :] = BRi[i]:dx:(BRi[i+1] + 0.01*dx)
14     end
15 =#

```

Following that various wavevectors, coefficients and transfer matrices that are initialised for later use.

```

1  #=
2      #Defining the wavevectors at regions '1' 'v' 'v+1' and 'n' by taking the
           mean of the wavevectors at each point in each region
3      kn = k(m,E,U(mean(Ri[N, :]),V))
4      k1 = k(m,E,U(mean(Ri[1, :]),V))
5      kv = k(m,E,U(mean(Ri[v, :]),V))
6      kvminus1 = k(m,E,U(mean(Ri[v-1]),V))
7
8      AV = 1.0
9      BV = 1.0
10     AVminus1 = 1.0
11     BVminus1 = 1.0
12     ANBN = [An ; Bn]
13     T1N = [1.0 0.0; 0.0 1.0]
14     TVN = [1.0 0.0; 0.0 1.0]
15     TVminus1N = [1.0 0.0; 0.0 1.0]
16     TE = ones(N-1)
17 =#

```

In the following piece of code the iterative wavevector k_i for each region is formed using the middle of the domain for each region (by taking the mean) to calculate the average potential for each region. Using $T_{i,i+1}$ and the following if statements I define the transfer matrices for $T_{V,N}$ and $T_{V-1,N}$ to initialise the coefficients A_V and B_V as well as A_{V-1} and B_{V-1} respectively. Following, which I define the wavevectors ψ_v and ψ_{v-1} with all the defined variables.

```

1  #=
2      #Forms the T1N (transfer matrix) from region 1 to the nth region
3      for i in 1:(N-1)
4          ki = k(m,E,U(mean(Ri[i, :]),V))
5          kiplus1 = k(m,E,U(mean(Ri[i+1, :]),V))
6          Tiplus1 = transferMatrix(ki,kiplus1,BRi[i+1])
7          T1N = T1N * Tiplus1
8
9          #Creates a transfer matrix for regions n to v, to help create a
          wavefunction at an arbitrary region v and same for region v+1
10         if i >= (v)
11             TVN = TVN * Tiplus1
12             #println(TVN)
13         end
14
15         if i >= (v-1)
16             TVminus1N = TVminus1N * Tiplus1
17             #println(TVminus1N)
18         end
19
20         if i == (N-1)
21             AVBV = TVN * ANBN
22             AV = AVBV[1]
23             BV = AVBV[2]
24
25             AVminus1BVminus1 = TVminus1N * ANBN
26             AVminus1 = AVminus1BVminus1[1]
27             BVminus1 = AVminus1BVminus1[2]
28         end
29     end
30  =#

```

Here I define the transmission and reflection probabilities using the equations I previously defined. I used secondary equations for both ($Tr1$ and $R1$) which rely on the elements of the $T_{1,N}$ matrix. I use them as a failsafe to ensure they are approximately the same ensuring there are no errors. They were not used here, I used them to test the accuracy of the probabilities.

```

1  #=
2  Transmission Probability
3  Tr = (An/A1)^2 * (kn/k1)
4  Tr1 = (1/(T1N[1][1])^2)*(kn/k1)
5  Reflection Probability
6  R = (B1/A1)^2
7  R1 = (T1N[2][1]/T1N[1][1])^2
8  =#

```

Finally I defined the transmission as a function of energy, essentially identical to that used for the finite barrier.

waveFunction4 (generic function with 1 method)

```

1 function
  waveFunction4(m::Float64,E::Float64,AN::Float64,BN::Float64,x1::Float64,
2  xnsb1::Float64,N::Int64,v::Int64,V::Float64,RorIorTr::String)
3
4      if v > N
5          return md"#### The specified region 'v' to create the wavefunction
6          is more than the total number of regions"
7      elseif v == 1
8          return md"#### The specified region 'v' to create the wavefunction
9          has to be greater than 1 to compare it to a region one less"
10     end
11
12     if N < 2
13         println("You need to input at least two regions")
14     else
15         #Defining the different gaps of the regions
16         xnsb1 = xnsb1
17         x1 = x1
18
19         deltax = (xnsb1 - x1)/(N-2)
20         deltax = truncateToSf(deltax, 3)
21
22         #Region boundaries
23         BRi = (x1 - deltax):deltax:(xnsb1 + deltax)
24
25         #Creation of regions
26         dx = 0.01
27         y = Int(round(((BRi[2] - BRi[1])/dx)) + 1)
28         Ri = ones(Float64, N , y)
29
30         for i in 1:(N)
31             Ri[i, :] = BRi[i]:dx:(BRi[i+1] + 0.01*dx)
32         end
33
34         #Defining the wavevectors at regions '1' 'v' 'v+1' and 'n' by taking
35         the mean of the wavevectors at each point in each region
36         kn = k(m,E,U(mean(Ri[N, :]),V))
37         k1 = k(m,E,U(mean(Ri[1, :]),V))
38         kv = k(m,E,U(mean(Ri[v, :]),V))
39         kvminus1 = k(m,E,U(mean(Ri[v-1]),V))
40
41         AV = 1.0
42         BV = 1.0
43         AVminus1 = 1.0
44         BVminus1 = 1.0
45         ANBN = [AN ; BN]
46         T1N = [1.0 0.0; 0.0 1.0]
47         TVN = [1.0 0.0; 0.0 1.0]
48         TVminus1N = [1.0 0.0; 0.0 1.0]
49         TE = ones(N-1)
50
51         #Forms the T1N (transfer matrix) from region 1 to the nth region
52         for i in 1:(N-1)
53             ki = k(m,E,U(mean(Ri[i, :]),V))
54             kiplus1 = k(m,E,U(mean(Ri[i+1, :]),V))
55             Tiplus1 = transferMatrix(ki,kiplus1,BRi[i+1])
56             T1N = T1N * Tiplus1

```



```

54
55     #Creates a transfer matrix for regions n to v, to help create a
56     wavefunction at an arbitrary region v and same for region v+1
57     if i >= (v)
58         TVN = TVN * Tiplus1
59     end
60
61     if i >= (v-1)
62         TVminus1N = TVminus1N * Tiplus1
63     end
64
65     if i == (N-1)
66         AVBV = TVN * ANBN
67         AV = AVBV[1]
68         BV = AVBV[2]
69
70         AVminus1BVminus1 = TVminus1N * ANBN
71         AVminus1 = AVminus1BVminus1[1]
72         BVminus1 = AVminus1BVminus1[2]
73     end
74
75     #Test to see if array is defined without any 'NaN' and 'Inf' values
76     #=
77     if isArrayDefined(xv)
78         println("All values in the array are defined and not NaN or Inf.")
79     else
80         println("The array contains NaN or Inf values.")
81     end
82     =#
83
84     psivminus1 = AVminus1.*exp.(i*kvminus1.*Ri[v-1, :]) .+ BVminus1.*exp.
85     (-i*kvminus1.*Ri[v-1, :])
86     psiv = AV.*exp.(i*kv.*Ri[v, :]) .+ BV.*exp.(-i*kv.*Ri[v, :])
87
88     A1B1 = T1N * ANBN
89     A1 = A1B1[1]
90     B1 = A1B1[2]
91
92     #Transmission Probability
93     Tr = (AN/A1)^2 * (kn/k1)
94     #Tr1 = (1/(T1N[1][1])^2)*(kn/k1)
95     #Reflection Probability
96     R = (B1/A1)^2
97     #R1 = (T1N[2][1]/T1N[1][1])^2
98
99     #To create the x (Energy(E)) and the y (Transmission(TE))
100     dE = evtoJ(0.01)
101     E1 = dE:dE:evtoJ(10.0)
102     a = Int(E1[end]/dE)
103     TE = ones(a)
104     for i in 1:a
105         kappa = k(m,E1[i],U(mean(Ri[2, :]), V))
106         kprime = k(m,E1[i],U(mean(Ri[2, :]), V))
107         ka = k(m,E1[i],0.0)
108
109         if E1[i] < U(mean(Ri[2, :]), V)

```

```

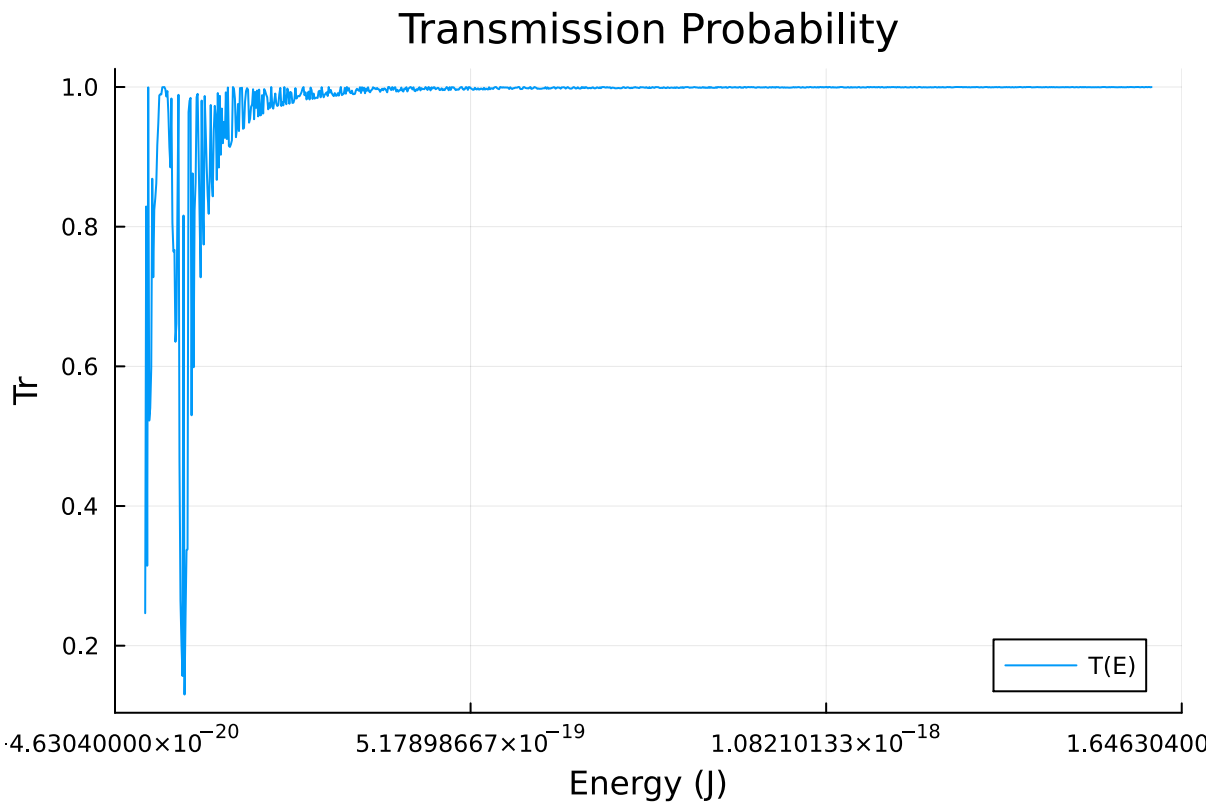
110         TE[i] = 1/(1 + (0.25)*(kappa/ka + ka/kappa)^(2)*
111         (sinh(kappa*deltax))^2)
112     elseif E1[i] == U(mean(Ri[2, :]), V)
113         TE[i] = 1/(1 + (ka*deltax/2)^2)
114     elseif E1[i] > U(mean(Ri[2, :]), V)
115         TE[i] = 1/(1 + (0.25)*(kprime/ka - ka/kprime)^(2)*
116         (sin(kprime*deltax))^2)
117     end
118 end

119 if RorIorTr == "Real"
120     #real
121     plot(Ri[v-1, :] ,real(psivminus1) ,xlabel= "x",ylabel = "psi",
122     label= "psivminus1" )
123     plot!(Ri[v, :] ,real(psiv) ,xlabel= "x",ylabel = "psi", label=
124     "psiv")
125     plot!(title="Wave function (real) in regions $(v-1) & $(v)")
126
127 elseif RorIorTr == "Imag"
128     #imaginary
129     plot!(Ri[v-1, :] ,imag(psivminus1) ,xlabel= "x",ylabel = "psi",
130     label= "psivminus1" )
131     plot!(Ri[v, :] ,imag(psiv) ,xlabel= "x",ylabel = "psi", label=
132     "psiv")
133     plot!(title="Wave function (imaginary) in region $(v-1) & $(v)")
134
135 elseif RorIorTr == "Tr"
136     #Transmission probability against the energy
137     plot(E1 ,TE ,xlabel= "Energy (J)",ylabel= "Tr", label= "T(E)")
138     plot!(title="Transmission Probability")
139
140 elseif RorIorTr != "Real" || "Imag" || "Tr"
141     md"##### Input either the string 'Real' or 'Imag' to plot either
142     the real or imaginary components of the wavefunction. And 'Tr' to
143     plot the transmission as a function of energy"
144 end

```

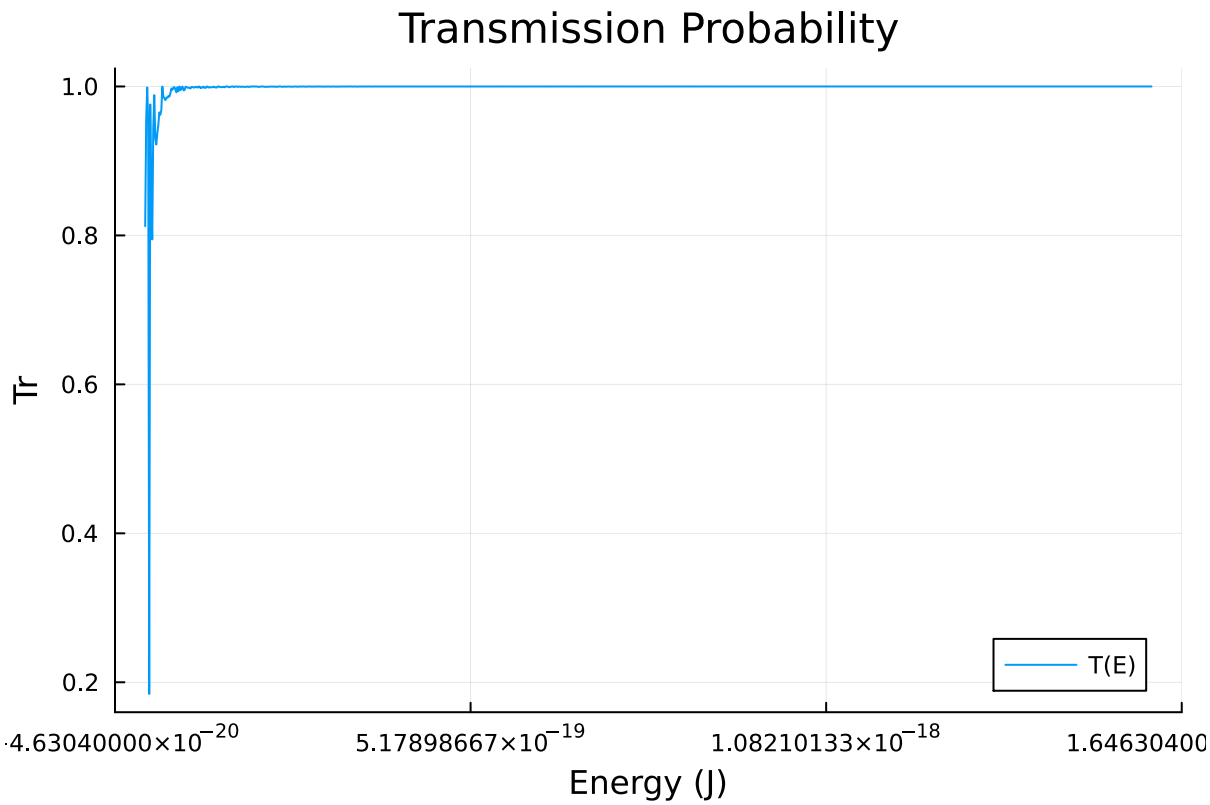
Syntax to call the function: $\text{waveFunction4}(m, E, A_N, B_N, x_1, x_{N-1}, N, v, V, \text{RorIorTr})$

Below (4.1) I have plotted the five-piece approximation for a smooth repelling Gaussian potential. Refer to [1] - Figure 14 (Page 48). In comparison to the literature the plot does not have a slight dip after it levels off. The plot shows a steep slope immediately then levels off and does not follow the expected increasing then decreasing slope with a point of inflection at the value of $V = 5eV$. This seems to follow the upper bound of the transmission probability for the finite barrier (2.2) as opposed to the expected lower bound.



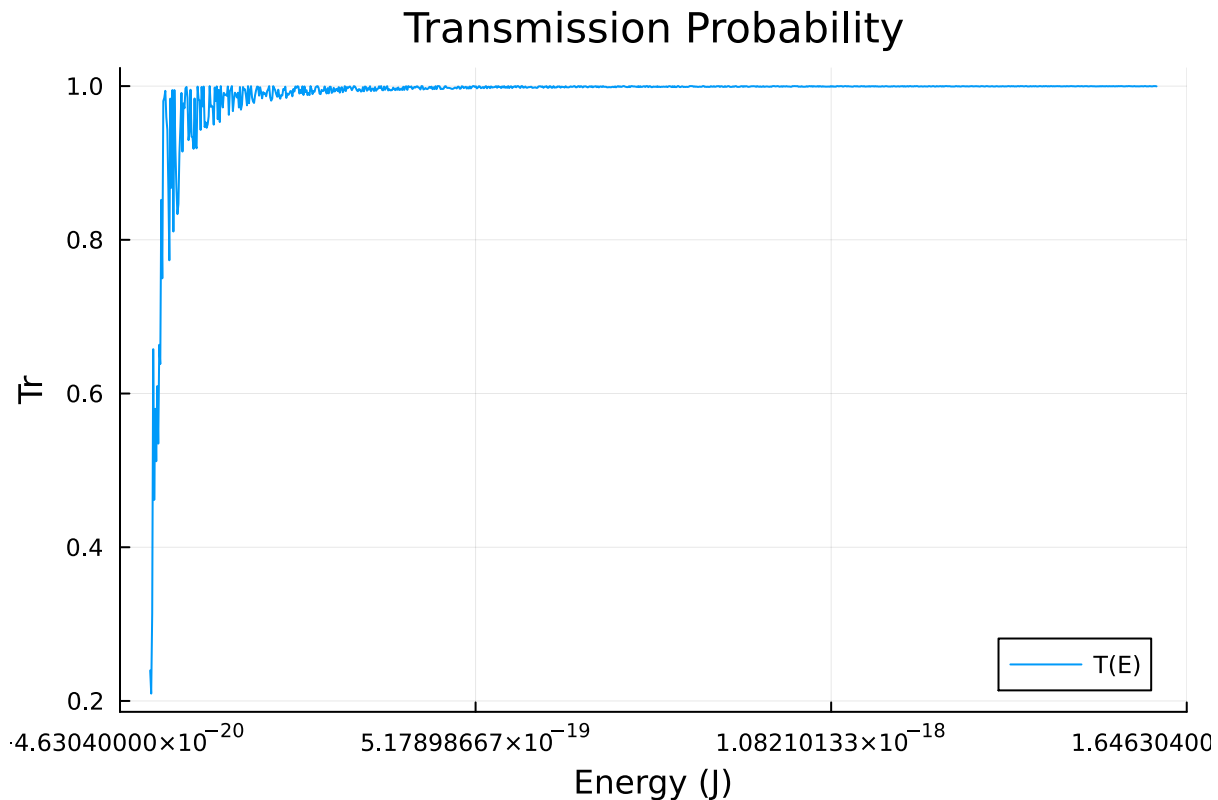
(Figure 4.1)

Here I have plotted the eleven-piece approximation for the same system. Refer to [1] - Figure 15 (Page 49). This shouldn't have the dip as expected however similarly it seems to follow the upper bound as opposed to the lower bound of the finite barrier once again as expected.



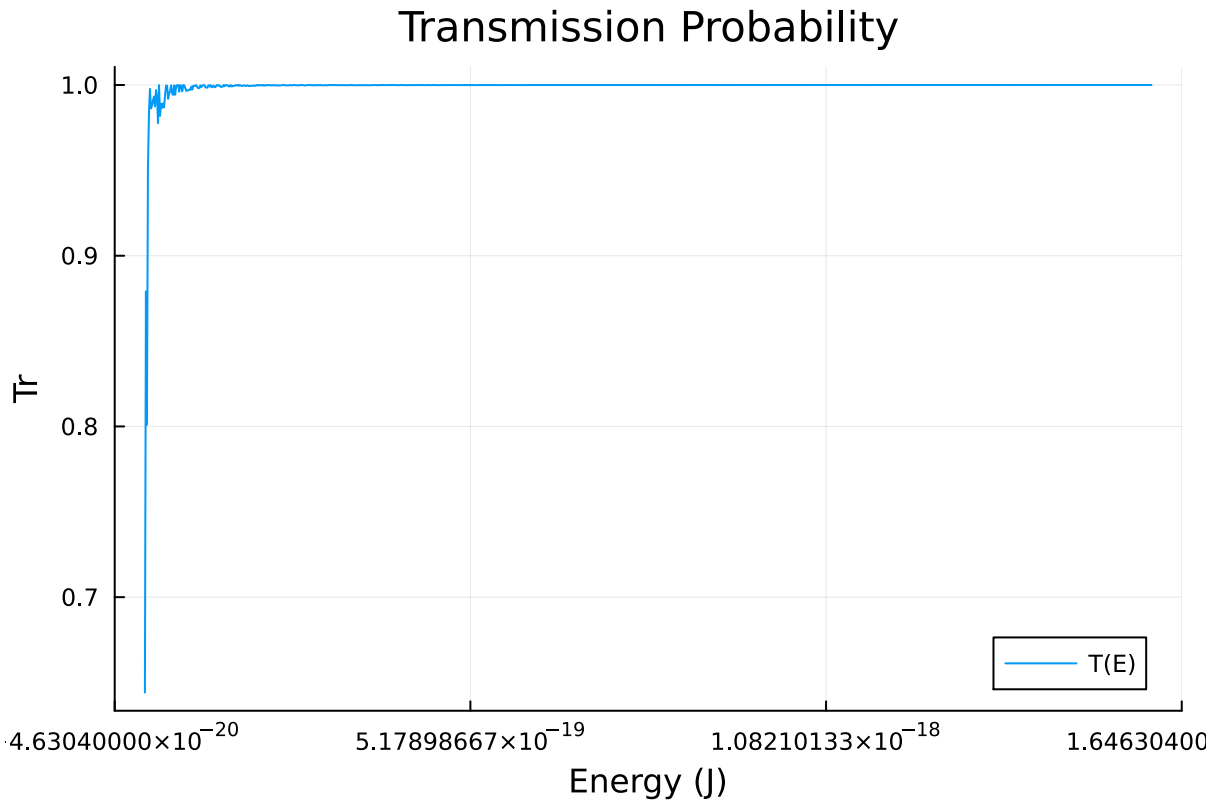
(Figure 4.2)

The five-piece approximation of an attracting Gaussian potential for when $V = -5eV$. Refer to [1] - Figure 16 (Page 50). This shape follows the literature far more closely, yet it still does not have the expected dip. As it is an attracting barrier, the electron's energy is not relevant even if the energy of the electron is less than that of the potential barrier.



(Figure 4.3)

This is the eleven-piece approximation for the same system. This follows the exact shape as expected. Without the the dip as shown in the literature, refer to [1] Figure 17 (Page 50).



(Figure 4.4)

Conclusions

To conclude, the biggest inaccuracies were in the lack of a dip after levelling off for the five piece approximations (4.1 & 4.3). Moreover, the two plots for the repelling barriers (4.1 & 4.2) did not pass $T_r = 0.5$ when $E = V$ (which in this case, $V = 5\text{eV}$). This should be the case as for a repelling barrier it's transmission can only be greater than it's reflection when the energy of the particle is greater than the potential barrier.

However, for the eleven-piece attracting Gaussian potential the plot is modelled exactly how the electron would behave for a range of energies. There were issues with the rounding and accuracy of values in Julia which made it difficult to define the boundary values and the regions. I also chose to take the variable potentials from the center of each region due to how it was modelled in reference [1].

The lack of accuracy in the transmission probability graphs for the arbitrary number of regions (Section 4) comes from the analytical nature of the equations (shown in Section 2). There are many limitations, analytical solutions tend not to work well for more complex systems. Take the double barrier example (Section 3). The equations I defined in Section 2 assume that it is a single rectangular potential barrier, where the potential on the left of the barrier is equal to the potential on the right $V_L = V_R$. If U_1 and U_2 are different it would be a poor approximation.

This is further exasperated for the Gaussian potential (Section 4) when using rectangular barriers as an approximation (five and eleven rectangle approximation). If I were to do this again I would tailor the solutions for each system. I would also select approximations which more closely follow the variable potential (i.e. a far greater number of regions which slowly get closer to the shape of the potential).

References

[1] = Robert Gilmore (2013) *Elementary Quantum Mechanics in One Dimension*

[2] = Wikipedia Contributors (2023) *Resonances in scattering from potentials*

[3] = (2013) *4.4 The Potential Step*

[4] = Nikola Opacak, Vitomir Milanovic and Jelena Radovanovic (2017) *Transmission singularities in resonant electron tunneling through double complex potential barrier*